CS220 Lab#9
32-bit MIPS Processor
(Part-3: Keyboard input, lui, load/store)

Mainak Chaudhuri
Indian Institute of Technology Kanpur

Sketch

- In this lab, you will incorporate lui and aligned load/store instructions (lw, lb, lh, lbu, lhu, sw, sb, sh) in the three-cycle MIPS processor design from the previous lab
- Assignment#1: Incorporate support to take input values from keyboard
  - Carries two marks
- Assignment#2: Incorporate lui, lw, lb, lh, lbu, lhu, sw, sb, sh instructions
  - Carries five marks

Assignment#1

- Accepting input values from keyboard
  - System call instruction is used to communicate with the environment
    - So far, syscall has been used to display outputs (with syscall number SYS_write) and exit program (with syscall number SYS_exit)
    - Include a new syscall number SYS_read in defs.vh for reading from keyboard

      `define SYS_read 32'd1003
    - Recall that the syscall instruction takes two register operands, say, regfile[rs] and regfile[rd] where regfile[rs] contains the syscall number and for SYS_read, the input value should be written to regfile[rd]
      - rt and shift amount are not used (can be set to zero)

Assignment#1

- Accepting input values from keyboard
  - System call instructions execute in the second cycle
  - If the syscall number is 1003, the Processor module generates a new output signal *waiting_for_input* and stalls
  - The Computer module conveys the *waiting_for_input* signal to the environment
  - The environment responds by sending a 32-bit *input_value* and an *input_value_valid* bit through two new inputs to the Computer module which are relayed to the Processor module

Assignment#1

- Accepting input values from keyboard
  - The Processor module, on seeing *input_value_valid* become 1, resets *waiting_for_input* to 0
  - The Processor module comes out of stall when *input_value_valid* becomes 0 and completes the execution of the syscall instruction which copies *input_value* into the destination register of the syscall instruction
  - Implement the necessary logic in the Processor module, update the input/output ports of the Processor and Computer modules

Assignment#1

- Simulate the design using a testbench that implements the translation of the following

```
int x, y, z;
input x;          // addi $1, $0, 1003; syscall $1, $2
input y;          // syscall $1, $3
z = x + y;        // add $4, $2, $3
print z;          // addi $1, $0, 1004; syscall $1, $4
exit();           // addi $1, $0, 1001; syscall $1, $0
```

  - In the testbench, you cannot read from the keyboard
    - Set *input_value* to some value and *input_value_valid* to 1 when *waiting_for_input* becomes 1
    - Set *input_value_valid* to 0 when *waiting_for_input* becomes 0

Assignment#1

- Synthesize and implement the design
- Generate bit stream, export the hardware
- Write a Vitis IDE C program to run the C program given in the next slide on the design
  - You can borrow most of the MIPS machine code from the final program of the previous lab's Assignment#2

Assignment#1

- C program to run on the design

```
int i, x, N;
input x;
input N;
for (i=0; i<N; i++) {
   if ((i & 0x1) == 0) {
      x += f(x, i);
      print x;
   }
   else {
      x –= f(x, i);
      print x;
   }
}
exit ();
int f (int a, int b) { return a+b; }
```

Assignment#1

- The structure of the Vitis C program is shown in the next few slides
  - Note: the inputs entered from keyboard will not appear on the screen
- Show the output of your Vitis C program to the TAs to receive marks
  - The TA will tell you the input values of x and N
  - The printed values must match the expected output for those input values of x and N

Assignment#1

- Structure of Vitis C program

```
unsigned reset=1, ins_addr, ins, done_storing=0, done,
total_cycles, proc_cycles;
unsigned waiting_for_input, input_value_valid=0;
int i, x, N, input_value;
int out[4];
unsigned flush_io_regs, done_copying_io_regs=0,
print_count, print_out_count=1;
Xil_Out32(base_addr, reset);
Xil_Out32(base_addr+3, done_storing);
Xil_Out32(base_addr+4, done_copying_io_regs);
Xil_Out32(base_addr+6, input_value_valid);
reset=0;
Xil_Out32(base_addr, reset);
// Input the translated instruction sequence
```

Assignment#1

- Structure of Vitis C program

```
done_storing = 1;
Xil_Out32(base_addr+3, done_storing);
while (1) {
   flush_io_regs = 0;
   done = 0;
   waiting_for_input = 0;
   while (!flush_io_regs && !done &&
!waiting_for_input) {
      flush_io_regs = Xil_In32(base_addr+14);
      done = Xil_In32(base_addr+7);
      waiting_for_input = Xil_In32(base_addr+16);
   }
```

Assignment#1

- Structure of Vitis C program

```
if (waiting_for_input) {
   xil_printf("Enter input:\n\r");
   scanf("%d", &input_value);
   Xil_Out32(base_addr+5, input_value);
   input_value_valid = 1;
   Xil_Out32(base_addr+6, input_value_valid);
   input_value_valid = 0;
   Xil_Out32(base_addr+6, input_value_valid);
}
```

Assignment#1

- Structure of Vitis C program

```
if (flush_io_regs) {
   out[0] = Xil_In32(base_addr+8);
   out[1] = Xil_In32(base_addr+9);
   out[2] = Xil_In32(base_addr+10);
   out[3] = Xil_In32(base_addr+11);
   done_copying_io_regs = 1;
   Xil_Out32(base_addr+4, done_copying_io_regs);
   done_copying_io_regs = 0;
   Xil_Out32(base_addr+4, done_copying_io_regs);
   for (int i=0; i<4; i++) {
      xil_printf("out%d=%d\n\r", print_out_count, out[i]);
      print_out_count++;
   }
}
```

Assignment#1

- Structure of Vitis C program

```
        if (done) {
            print_count = Xil_In32(base_addr+15);
            int i=8;
            while (print_count > 0) {
               out[0] = Xil_In32(base_addr+i);
               xil_printf("out%d=%d\n\r", print_out_count, out[0]);
               print_count--;
               print_out_count++;
               i++;
            }
            break;
        }
    }  // while(1) loop ends
    total_cycles = Xil_In32(base_addr+12);
    proc_cycles = Xil_In32(base_addr+13);
    xil_printf("Total cycles: %u, computation cycles: %u\n\r",
    total_cycles, proc_cycles);
    printf("Processor cycles per instruction (CPI): %f\n\r",
    ((float)proc_cycles)/(ins_addr+1));
```

Assignment#2

- Incorporate the lui instruction in the ALU module
  - Opcode=0xf
    - I format encoding
    - rs not used
- Incorporate the aligned load and store instructions assuming big-endian memory
  - Opcodes in parentheses: lb (0x20), lh (0x21), lw (0x23), lbu (0x24), lhu (0x25), sb (0x28), sh (0x29), sw (0x2b)
    - I format encoding
    - Immediate is sign-extended

Assignment#2

- Incorporate the aligned load and store instructions assuming big-endian memory
  - The address computation should be implemented in the ALU module
    - Note that this is a byte address
    - Use *branch_offset* input to pass sign-extended immediate
    - Use *dest* output to emit the computed address
    - Set *dest_valid* to 1 for load instructions because these instructions will write to the register file
    - Set *dest_valid* to 0 for store instructions
  - The ALU module should continue to retain its combinational always structure

Assignment#2

- Incorporate the aligned load and store instructions using big-endian memory
  - The Processor module uses four new outputs to talk to the Memory module for doing load/store operations
    - 8-bit *data_addr*: carries the address of load/store
      - Word address derived from the byte address coming out of ALU
    - 1-bit *data_addr_valid*: set to 1 only for load/store instructions in the second cycle
    - 2-bit *data_mem_command*: set to `READ_COMMAND for loads, `WRITE_COMMAND for sw, `SUBWORD_WRITE_COMMAND for sb, sh
      - Define these in defs.vh, update necessary modules to reflect the changed width of memory command (previously it was 1 bit wide)
    - 32-bit *store_value*: the value to be stored in memory

Assignment#2

- Incorporate the aligned load and store instructions using big-endian memory
  - The Processor module uses a new 32-bit input, say, *load_value* to receive the word from memory in response
  - For lw instruction, *load_value* can be sent to the register file for writing
  - For lb, lbu instructions, the correct byte must be extracted from *load_value* and sent to register file for writing after sign/zero extension
  - For lh, lhu instructions, the correct half-word must be extracted from *load_value* and sent to register file for writing after sign/zero extension

Assignment#2

- Incorporate the aligned load and store instructions using big-endian memory
  - For sb, sh instructions, the word at the given address must be read first, then the appropriate bytes must be modified and the modified word should be written to memory
  - To implement this, modify the Memory module to respond with the word (*load_value*) when the command is `READ_COMMAND or `SUBWORD_WRITE_COMMAND
  - Modify the Memory module to write when the command is `WRITE_COMMAND or `SUBWORD_WRITE_COMMAND

Assignment#2

- Incorporate the aligned load and store instructions using big-endian memory
  - For sb, sh instructions, the 32-bit *store_value* must be prepared using the 32-bit *load_value* and the contents to be stored
    - For sb, the least significant byte of regfile[rt] is to be stored and for sh, the least significant half word of regfile[rt] is to be stored
    - Which bytes of the 32-bit *load_value* should be modified with the bytes to be stored depend on the address computed by ALU
      - For example, an sb instruction to address 2, would copy the least significant byte (call it *v*) of regfile[rt] into *load_value*[15:8] leaving other bytes of *load_value* unchanged i.e., in this case, *store_value* = {*load_value*[31:16],*v*,*load_value*[7:0]};

Assignment#2

- Incorporate the aligned load and store instructions using big-endian memory
  - The Memory module continues to have a single read port and a single write port
    - You should not add any additional input/output to Memory module
    - The read port is shared between data fetch and instruction fetch
    - The write port is shared between instruction stores (from environment) and data stores
    - Update the Computer module to generate the correct arguments for the Memory module

Assignment#2

- Incorporate the aligned load and store instructions using big-endian memory
  - The ALU module computes byte address
  - The Processor module fetches the word at the word address and computes the word to be loaded in the destination register for load instructions and the word to be stored in the memory for store instructions
- Write a simulation testbench to test the individual load/store instructions
  - Recall that we have a 1024-byte memory (256 words); several words starting from address 0 are used to store code; rest can be used for data

Assignment#2

- To test all load/store instructions and lui, you may do the following
  - Store a 32-bit constant (say, 0xfe7654dc) starting at address 512 using lui, ori, sw
  - Load the word using lw and print it
  - Load four different bytes using lb and lbu, and print the destination register contents
  - Load two different half words using lh and lhu, and print the destination register contents
  - Store new byte values using four sb instructions, load the word using lw, and print it
  - Store new half word values using two sh instructions, load the word using lw, and print it

Assignment#2

- Synthesize, implement, generate bit stream, export hardware, and write a Vitis IDE C program to run the programs given in the next three slides
  - Follow the structure of the Vitis C program as used in Assignment#1
  - The outputs should match with the expected results
- If your design fails to meet timing, examine the critical path and optimize the design
  - For our small memory of 1KB, address computation, memory lookup, and computation of word to be loaded or stored can be accommodated in a cycle

Assignment#2

- Program#1

```
unsigned char A[]="Hello World!\n";
for (int i=0; i<13; i++) {
   print A[i];
}
exit();
```

  - Notes:
    - Assume that array A starts at byte address 512
    - Assemble the ASCII values of the characters in "Hello World!\n" and store them in memory starting at address 512 using 3 lui, 3 ori, 3 sw, 1 addi, 1 sb
    - Printing A[i] is accomplished by first loading the value stored in A[i] in a register using lbu instruction and then executing a print syscall as usual
    - In the Vitis C program, use "%c" in place of "out%d=%d\n\r" for printing the out[] values

Assignment#2

- Program#2

```
int x, N, i, y=0;
input x;
input N;
short *A; // Assume that A starts at address x and
          // has N elements
if (x < 512) print 0;   // A not in legitimate address range
else if ((x & 0x1) != 0) print 0;  // A not aligned
else if ((x+(N << 1)) > 1024) print 0;  // A too large
else { // A can be allocated
   for (i=0; i<N; i++) {
      if ((i & 0x1) == 0) A[i] = i;  // Use sh
      else A[i] = –i;  // Use sh
    }
    for (i=0; i<N; i++) y += A[i];   // Use lh
    print y;
}
exit();
```

Assignment#2

- Program#3

```
int n, x;
input n;
if (n < 0) print –1;
else {
   x = sum(n);
   print x;
}
exit();
int sum (int n) {
   if (n == 0) return 0;
   return n+sum(n-1);
}
```

  - Initialize $sp to 1024 in the beginning
  - Find out the largest n for which it works; reason?